

Week 4 Resources: Training PPO on the Trading Environment

Goal of Week 4

Week 3 ended with two things you built: a PPO agent that reliably solves CartPole, and a toy trading environment that you ran with a random agent. Those two pieces were separate.

Week 4 connects them. You will train PPO on your own trading environment, improve the state representation to give the agent more useful information, and evaluate the result properly — going beyond just “does the reward go up?” to asking “does the agent actually make money?”

By the end of this week you should be able to:

- Train a PPO agent on a custom Gymnasium environment you wrote yourself.
- Explain what a trading agent’s state, action, and reward mean, and how changing any of them changes what the agent can learn.
- Add a window of recent price returns to the state and explain why that helps.
- Add a transaction cost to the reward and describe how the agent’s behaviour changes.
- Plot and interpret a cumulative P&L curve and compare it against a random baseline and a buy-and-hold baseline.

Recap: Key Ideas from Week 3

Before moving on, confirm you can answer these:

- What does your `ToyTradingEnv` return as observation, action, and reward?
- Why did PPO produce a smoother reward curve than REINFORCE?
- What is the clipping constraint in PPO, and what problem does it prevent?
- What is the difference between `n_steps` and `batch_size` in Stable-Baselines3?

If any of these feel shaky, re-read the Week 3 resources before continuing.

From CartPole to a Trading Environment

CartPole is a well-understood benchmark: everyone knows what “solved” looks like (average return ≥ 475 over 100 episodes), the physics are deterministic, and you can visually verify the agent is working.

Your trading environment is different in every one of those ways:

- **There is no obvious “solved” threshold.** A random walk has no exploitable structure. The agent cannot reliably beat zero — and that is expected and informative, not a bug.
- **The dynamics are stochastic and memoryless.** Tomorrow’s price return is independent of today’s. This is the core property of an efficient market, and it means the agent cannot learn a pattern that does not exist.

- **What success looks like is ambiguous.** Is “beat the random baseline” enough? Is a consistent Sharpe ratio of 0.2 good? You have to define your own evaluation criterion.

This week, you will start to feel the difference between “RL working correctly” and “RL finding a profitable strategy.” On a random walk, a well-functioning agent should converge to roughly zero — that is the right answer. If it appears to find profit, be suspicious: it may be overfitting to noise in the training episodes.

Improving the State: A Price History Window

In Week 3, your state was two numbers: `[last_price_return, current_position]`. This is the minimum possible information. The agent can see what just happened and where it is, but nothing more.

A natural extension is to give the agent a **window** of the last k price returns instead of just the most recent one. The observation then becomes:

$$[r_{t-k+1}, r_{t-k+2}, \dots, r_{t-1}, r_t, \text{current_position}],$$

a $(k + 1)$ -dimensional vector. The agent can now see short-term momentum — or lack of it.

Does this help on a random walk? In theory, no: past returns contain no information about future returns on a true random walk. In practice, the agent may still learn something about *position management* (e.g., “I am long and prices have been falling — I should cut”). That is a valid thing to learn even when the price series has no exploitable trend.

The key lesson: **a bigger state is not always better.** More dimensions mean a harder optimisation problem, more data needed, and more opportunity to overfit. Always start with the minimum state that is *conceptually* sufficient, then add features one at a time.

Implementing a Window in Gymnasium

The observation space must change to reflect the new state dimension. If you add k historical returns plus the current position:

```
obs_dim = k + 1 # k returns + current position
self.observation_space = spaces.Box(
    low=np.full(obs_dim, -np.inf, dtype=np.float32),
    high=np.full(obs_dim, np.inf, dtype=np.float32),
)
```

You also need to maintain a price history buffer inside `reset()` and `step()`. A `collections.deque` with `maxlen=k` is convenient: it automatically drops the oldest entry when a new one is appended.

Transaction Costs and Reward Shaping

The Week 3 trading environment rewarded the agent purely on `position * price_return * 100`. There was no cost for changing position. This creates a subtle problem: the agent might *churn* — flipping between long and short every step — because there is no penalty for doing so. In real markets, every trade incurs a cost (bid-ask spread, commissions, market impact).

Adding a **transaction cost** modifies the reward:

$$\text{reward} = \text{position} \times \text{price_return} \times 100 - \text{cost} \times |\text{new_position} - \text{old_position}|,$$

where **cost** is a small positive constant (e.g. 0.1). The cost is paid only when the position *changes*, so holding is free. This discourages excessive trading.

Reward shaping — modifying the reward function to encourage desired behaviour — is one of the most powerful (and most dangerous) tools in applied RL. Powerful because it can dramatically speed up learning. Dangerous because a poorly designed reward can cause the agent to find unintended shortcuts.

A useful test: ask yourself, “If the agent found a policy that maximised this reward but did nothing I intended, what would it look like?” For the transaction-cost reward above, the worst-case unintended policy might be to always hold flat (zero position) — earning zero on every step with zero transaction cost. Whether that is acceptable depends on your goals.

Evaluating a Trading Agent Properly

A reward curve (episode return vs. training episode) tells you whether the agent is *learning*. It does not tell you whether the agent is doing something *useful*. For a trading environment, you need at least two additional evaluation tools.

Cumulative P&L Curve

Run the trained agent on a fresh episode (not used during training) and plot its cumulative profit and loss (P&L) over time:

```
pnl = 0.0
pnl_history = []
for step in range(episode_length):
    action, _ = model.predict(obs)
    obs, reward, done, _, _ = env.step(action)
    pnl += reward
    pnl_history.append(pnl)
```

Plot `pnl_history` alongside the same curve for a random agent. If the trained agent’s curve is consistently above the random agent’s, it has learned something. If they are indistinguishable, the agent has learned nothing useful — and that is *also* useful information about the environment.

Buy-and-Hold Baseline

A buy-and-hold agent always stays long (position = +1) and never trades. Its reward at each step is simply $\text{price_return} \times 100$. On a symmetric random walk, this baseline has an expected return of zero, but it serves as a sanity check: any trained agent that consistently underperforms buy-and-hold is actively destroying value.

Why Near-Zero Is the Right Answer Here

On a pure random walk, the expected cumulative P&L of *any* strategy is zero. The random agent, buy-and-hold, and a well-trained PPO agent should all converge to near-zero average P&L

over many evaluation episodes. If your agent claims to earn consistently positive P&L, one of the following is true:

1. The environment’s random seed is not being reset properly — the agent memorised one episode.
2. The agent found a statistical artefact in a small sample.
3. You inadvertently introduced a bias in the reward function.

Learning to recognise and diagnose these issues is a core skill you are building this week.

Comparing REINFORCE and PPO Revisited

Last week you compared the two algorithms on CartPole. This week, think about what would happen if you tried REINFORCE on the trading environment:

- CartPole episodes are short (10–500 steps) and the reward signal is immediate (+1 per step).
- Trading episodes are longer (100 steps) and the reward signal is noisy (random walk returns fluctuate a lot).

REINFORCE estimates the gradient from a single trajectory. On a noisy reward signal, a single trajectory is a poor estimate of the gradient — the variance is enormous. PPO collects batches of experience (e.g. `n_steps=512` across multiple episodes) before updating, which averages out much of the noise.

This is one reason why PPO (and actor-critic methods in general) are almost always preferred over vanilla REINFORCE on real problems. The trading environment is a first taste of a “real problem” in this sense.

Connecting Back to the Five Questions

As always, ground the new ideas in the core RL framework:

- **Agent:** The PPO policy network, selecting Buy/Hold/Sell at each step.
- **Environment:** Your `ToyTradingEnv` (or the extended `v2` version with history window).
- **State:** $[r_{t-k+1}, \dots, r_t, \text{current_position}]$ — recent price returns plus position.
- **Actions:** Discrete — 0 = Go Long, 1 = Hold, 2 = Go Short.
- **Reward:** $\text{position} \times \text{price_return} \times 100 - \text{cost} \times |\Delta \text{position}|$ (with transaction cost).

Suggested External Resources (Optional)

You only need this document and the assignment to complete Week 4. If you want more context:

- Stable-Baselines3 documentation — Getting Started and Custom Environments sections.
- HuggingFace Deep RL Course — Unit 5 (custom environments and reward shaping).
- Sutton & Barto, *Reinforcement Learning: An Introduction*, Chapter 17 — a short discussion on reward shaping and its pitfalls.

Use external resources to reinforce your understanding, not as something to memorise.

Self-Check Before Moving On

You are ready for Week 5 when you can:

- Train PPO on your own `ToyTradingEnv` and produce a reward curve.
- Explain why the agent's average P&L should be near zero on a random walk.
- Add a price history window to the state and describe what information it gives the agent.
- Add a transaction cost and explain how it changes the agent's incentives.
- Plot and compare cumulative P&L curves for a trained agent, a random agent, and buy-and-hold.
- Look at a cumulative P&L curve and tell whether the agent learned something or nothing — and explain how you know.